

Resolução da Lista 2 - Métodos numéricos

Larissa de Souza e Otávio Menezes

05/10/2026

1 Questão 1 — Gerador de matrizes positivas definidas

1.1 Ideia

O teorema do enunciado diz que basta construir uma matriz que seja, *ao mesmo tempo*, (i) simétrica, (ii) estritamente diagonal dominante [Definição 6.20, p. 457], isto é, $|a_{ii}| > R_i$ para todo i , e (iii) com diagonal não-negativa. As três propriedades podem ser *impostas por construção*: sorteamos livremente a parte fora da diagonal (de forma simétrica) e, só depois, escolhemos cada a_{ii} maior que a soma R_i já sorteada.

Algorithm 1 gerar_matriz_pd(n)

Require: dimensão n

Ensure: $A \in \mathbb{R}^{n \times n}$ positiva definida

```
1: for  $i = 2, \dots, n$  do
2:   for  $j = 1, \dots, i - 1$  do
3:     sorteie  $a_{ij} \sim \mathcal{U}(-1, 1)$  e faça  $a_{ji} \leftarrow a_{ij}$                                  $\triangleright$  simetria
4:   end for
5: end for
6: for  $i = 1, \dots, n$  do
7:   sorteie  $\delta_i \sim \mathcal{U}(1, 2)$                                                      $\triangleright$  folga estritamente positiva
8:    $a_{ii} \leftarrow R_i + \delta_i = \sum_{j \neq i} |a_{ij}| + \delta_i$ 
9: end for
10: return  $A$ 
```

1.2 Por que *toda* matriz gerada é positiva definida

Por construção:

- (i) $a_{ji} = a_{ij}$, logo $A = A^\top$;
- (ii) $a_{ii} = R_i + \delta_i > R_i$ com $\delta_i \geq 1 > 0$, logo A é estritamente diagonal dominante;
- (iii) $a_{ii} = R_i + \delta_i \geq 1 > 0$, logo a diagonal é positiva (em particular não-negativa).

As hipóteses do teorema são satisfeitas para *qualquer* sorteio, portanto A é PD. Para deixar a justificativa completa, mostramos também o teorema, com uma estimativa quantitativa.

Teorema 1. *Se A é simétrica, estritamente diagonal dominante e $a_{ii} \geq 0$, então para todo $x \in \mathbb{R}^n$*

$$x^\top A x \geq \min_i (a_{ii} - R_i) \|x\|_2^2,$$

e, como $\min_i (a_{ii} - R_i) > 0$, A é positiva definida.

Demonstração. Da dominância estrita, $|a_{ii}| > R_i \geq 0$; como $a_{ii} \geq 0$, temos $a_{ii} = |a_{ii}| > R_i$. Expandindo a forma quadrática e usando $x_i x_j \geq -|x_i| |x_j|$:

$$x^\top A x = \sum_i a_{ii} x_i^2 + \sum_{i \neq j} a_{ij} x_i x_j \geq \sum_i a_{ii} x_i^2 - \sum_{i \neq j} |a_{ij}| |x_i| |x_j|.$$

Pela desigualdade $|x_i||x_j| \leq \frac{1}{2}(x_i^2 + x_j^2)$ e pela simetria ($|a_{ij}| = |a_{ji}|$),

$$\sum_{i \neq j} |a_{ij}| |x_i| |x_j| \leq \frac{1}{2} \sum_{i \neq j} |a_{ij}| x_i^2 + \frac{1}{2} \sum_{i \neq j} |a_{ji}| x_j^2 = \sum_i x_i^2 \sum_{j \neq i} |a_{ij}| = \sum_i R_i x_i^2.$$

Logo $x^\top Ax \geq \sum_i (a_{ii} - R_i) x_i^2 \geq \min_i (a_{ii} - R_i) \|x\|_2^2$, que é > 0 se $x \neq 0$. $\square$

Outras duas justificativas, usando apenas resultados do livro.

- **Via autovalores.** Pelo Teorema do Círculo de Geršgorin [Teorema 9.1, p. 626], todo autovalor λ de A pertence a algum disco $\{z : |z - a_{ii}| \leq R_i\}$. Como A é simétrica, seus autovalores são reais [Corolário 9.17, p. 639]; então $\lambda \geq a_{ii} - R_i = \delta_i \geq 1 > 0$. Uma matriz simétrica com todos os autovalores positivos é PD [Teorema 9.18, p. 639].
- **Via eliminação de Gauss.** Matrizes estritamente diagonal dominantes admitem eliminação de Gauss *sem trocas de linhas* e cada matriz reduzida continua estritamente diagonal dominante [Teorema 6.21, p. 458]. Com diagonal positiva, cada novo pivô satisfaz $a_{jj}^{(2)} = a_{jj} - \frac{a_{j1}a_{1j}}{a_{11}} \geq a_{jj} - |a_{j1}| \frac{|a_{1j}|}{a_{11}} > a_{jj} - |a_{j1}| > 0$ (a última desigualdade vale porque $a_{jj} > R_j \geq |a_{j1}|$). Assim, todos os pivôs são positivos e, pelo [Teorema 6.26, p. 463], a matriz simétrica A é PD.

Observação 1. Escolher $\delta_i \in [1, 2)$ (e não apenas $\delta_i > 0$) garante, pela estimativa acima, que o menor autovalor é ≥ 1 . Isso evita matrizes quase singulares e mantém o condicionamento bom, o que é desejável no experimento da Questão 2 com n até 500.

1.3 Três exemplos gerados (questao1.py, semente 2026)

Para cada exemplo, o programa verifica simetria, dominância diagonal estrita e diagonal não-negativa, calcula a folga $\min_i (a_{ii} - R_i)$ e, como teste empírico, o menor valor de $x^\top Ax / x^\top x$ entre 20 000 vetores aleatórios. Pelo Teorema 1, este último deve ser $\geq$ folga.

$$A_1 = \begin{bmatrix} 2.2925 & -0.6421 & 0.2798 \\ -0.6421 & 2.0625 & -0.0655 \\ 0.2798 & -0.0655 & 2.1358 \end{bmatrix} \quad A_2 = \begin{bmatrix} 3.1459 & -0.9081 & 0.8514 & -0.2326 \\ -0.9081 & 3.4099 & -0.4378 & 0.4900 \\ 0.8514 & -0.4378 & 3.2717 & -0.4743 \\ -0.2326 & 0.4900 & -0.4743 & 2.3351 \end{bmatrix}$$

$$A_3 = \begin{bmatrix} 4.1529 & -0.2612 & -0.9034 & -0.9409 & 0.8354 \\ -0.2612 & 2.2808 & -0.0447 & -0.0015 & 0.3924 \\ -0.9034 & -0.0447 & 3.7508 & -0.8454 & 0.4409 \\ -0.9409 & -0.0015 & -0.8454 & 4.0634 & -0.4299 \\ 0.8354 & 0.3924 & 0.4409 & -0.4299 & 3.2997 \end{bmatrix}$$

	simétrica	estr. diag. dom.	$a_{ii} \geq 0$	$\min_i (a_{ii} - R_i)$	$\min x^\top Ax / x^\top x$ (empírico)
A_1 (3×3)	sim	sim	sim	1.3549	1.4968
A_2 (4×4)	sim	sim	sim	1.1382	1.9436
A_3 (5×5)	sim	sim	sim	1.2012	1.7381

Tabela 1: Verificações dos exemplos. Por exemplo, em A_1 : $R = (0.9220, 0.7076, 0.3453)$ e $\text{diag}(A_1) = (2.2925, 2.0625, 2.1358)$.

2 Questão 2 — Eliminação de Gauss, $P^\top LU$ e Cholesky

2.1 Substituições (progressiva e regressiva)

Os três métodos terminam em sistemas triangulares.

- **Substituição regressiva** ($Ux = y$, U triangular superior) [Algoritmo 6.1, passos 8–9, p. 403]:

$$x_n = \frac{y_n}{u_{nn}}, \quad x_i = \frac{1}{u_{ii}} \left(y_i - \sum_{j=i+1}^n u_{ij} x_j \right), \quad i = n-1, \dots, 1.$$

- **Substituição progressiva** ($Ly = b$, L triangular inferior) [p. 450–451]:

$$y_1 = \frac{b_1}{l_{11}}, \quad y_i = \frac{1}{l_{ii}} \left(b_i - \sum_{j=1}^{i-1} l_{ij} y_j \right), \quad i = 2, \dots, n,$$

com $l_{ii} = 1$ (sem divisão) no caso de Doolittle.

- **Regressiva com a transposta** ($L^\top x = y$, usada no Cholesky) [passos 9–10, p. 466]: como $(L^\top)_{ij} = l_{ji}$, não é preciso montar $L^\top$: $x_i = \frac{1}{l_{ii}} (y_i - \sum_{j=i+1}^n l_{ji} x_j)$.

Cada substituição custa $\approx n^2/2$ multiplicações e $\approx n^2/2$ adições, isto é, $O(n^2)$.

2.2 Eliminação de Gauss com pivoteamento parcial com escala

A eliminação transforma $[A | b]$ numa matriz triangular superior com as operações $(E_j - m_{ji}E_i) \rightarrow (E_j)$, $m_{ji} = a_{ji}^{(i)} / a_{ii}^{(i)}$ [Seção 6.1, p. 396–403]. Um pivô pequeno em relação aos demais elementos amplifica erros de arredondamento (exemplo do pivô 0,003 em [Seção 6.2, p. 411–413]). O **pivoteamento parcial com escala** [Algoritmo 6.3, p. 416–417] compara os candidatos *relativamente* ao tamanho de suas linhas:

1. fatores de escala, calculados *uma única vez*: $s_i = \max_{1 \leq j \leq n} |a_{ij}|$ (se algum $s_i = 0$, não há solução única);
2. na etapa i , escolhe-se o menor $p \geq i$ tal que $\frac{|a_{pi}|}{s_p} = \max_{i \leq k \leq n} \frac{|a_{ki}|}{s_k}$ e troca-se $(E_i) \leftrightarrow (E_p)$;
3. eliminação: $m_{ji} = a_{ji}/a_{ii}$ e $E_j \leftarrow E_j - m_{ji}E_i$, $j > i$; ao final, substituição regressiva.

Como no livro, as trocas são *simuladas* com o vetor **NLINHA** (só índices são trocados). **Custo**: $\frac{n^3}{3} + n^2 - \frac{n}{3}$ multiplicações/divisões e $\frac{n^3}{3} + \frac{n^2}{2} - \frac{5n}{6}$ adições/subtrações [p. 406–407]; a escala acrescenta apenas $\frac{3}{2}n(n-1)$ comparações e $\frac{n(n+1)}{2} - 1$ divisões [Eq. (6.7), p. 419], ou seja, $O(n^2)$. **Ponto crucial para o item (b)**: a eliminação opera sobre $[A | b]$; para um novo b , todo o trabalho $O(n^3/3)$ precisa ser refeito.

2.3 Decomposição $P^\top LU$

Se a eliminação pode ser feita sem trocas, então $A = LU$, com L triangular inferior unitária formada pelos multiplicadores m_{ji} e U a matriz triangular final [Teorema 6.19, p. 448]. Havendo trocas, existe uma matriz de permutação P (identidade com linhas reordenadas) tal que $PA = LU$; como $P^{-1} = P^\top$,

$$A = P^\top LU \quad [\text{p. 451–452}].$$

Cálculo de L e U (forma de Doolittle, $l_{ii} = 1$) [Algoritmo 6.4, p. 450]: igualando as entradas de $PA = LU$,

$$u_{ij} = a_{ij} - \sum_{k=1}^{i-1} l_{ik} u_{kj} \quad (j \geq i), \quad l_{ji} = \frac{1}{u_{ii}} \left(a_{ji} - \sum_{k=1}^{i-1} l_{jk} u_{ki} \right) \quad (j > i).$$

Na etapa i , antes de dividir por u_{ii} , calculam-se os candidatos $a_{ji} - \sum_{k=1}^{i-1} l_{jk} u_{ki}$ ($j \geq i$), que são exatamente os valores $a_{ji}^{(i)}$ da eliminação, e escolhe-se o pivô pelo *mesmo* critério com escala; a troca de linhas é registrada no vetor **perm**, que representa P . **Resolução**: $Ax = b \iff LUx = Pb$; resolve-se $Ly = Pb$ (progressiva) e $Ux = y$ (regressiva). **Custo**: fatoração $O(n^3/3)$ (a mesma da eliminação), mas cada novo b custa apenas $O(2n^2)$ [Seção 6.5, p. 445–446]: “uma vez que a fatoração esteja determinada, os sistemas envolvendo a matriz A podem ser resolvidos desta maneira simplificada para qualquer número de vetores b ”.

2.4 Decomposição de Cholesky

Pelo [Corolário 6.28, p. 463], A é PD se e somente se $A = LL^\top$ com L triangular inferior de diagonal não nula. Igualando as entradas de $A = LL^\top$ (para $j \geq i$, $a_{ji} = \sum_{k=1}^i l_{jk} l_{ik}$) e isolando o termo $k = i$, obtêm-se as fórmulas do [Algoritmo 6.6, p. 464]:

$$l_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2}, \quad l_{ji} = \frac{1}{l_{ii}} \left(a_{ji} - \sum_{k=1}^{i-1} l_{jk} l_{ik} \right), \quad j = i+1, \dots, n.$$

Resolução: $Ly = b$ (progressiva) e $L^\top x = y$ (regressiva com a transposta). **Por que não precisa de pivoteamento:** para A simétrica PD, a eliminação sem trocas tem todos os pivôs positivos e é estável em relação ao crescimento de erros de arredondamento [Teorema 6.26, p. 463]. **Custo:** $\frac{n^3}{6} + \frac{n^2}{2} - \frac{2n}{3}$ multiplicações/divisões e $\frac{n^3}{6} - \frac{n}{6}$ adições/subtrações, mais n raízes quadradas [p. 465] — *metade* do custo de LU , porque a simetria permite calcular só a metade inferior.

Aplicabilidade. As matrizes do experimento vêm da Questão 1, logo são PD. Então A é invertível [Teorema 6.23(i), p. 460], de modo que a eliminação e a fatoração $P^\top LU$ existem, e a fatoração de Cholesky existe pelo Corolário 6.28. Os três métodos produzem a mesma solução; o programa confere o resíduo $\|Ax - b\|_\infty$, que ficou abaixo de 5×10^{-14} em todos os sistemas.

2.5 Desenho do experimento e previsão teórica

- (a) Para cada n , 10 pares (A_k, b_k) com A_k distintas: cada método processa cada A_k do zero.
- (b) Para cada n , uma única A e 10 vetores b_k : Gauss refaz a eliminação 10 vezes, enquanto $P^\top LU$ e Cholesky **fatoram uma única vez** e reutilizam os fatores, repetindo apenas as substituições.

Contando apenas os termos dominantes (multiplicações):

	Gauss (escala)	$P^\top LU$	Cholesky
(a) 10 matrizes	$10 \cdot \frac{n^3}{3}$	$10(\frac{n^3}{3} + n^2)$	$10(\frac{n^3}{6} + n^2)$
(b) 1 matriz, 10 vetores	$10 \cdot \frac{n^3}{3}$	$\frac{n^3}{3} + 10n^2$	$\frac{n^3}{6} + 10n^2$

Tabela 2: Custo teórico aproximado.

Previsão pela contagem de operações: em (a), Cholesky $\approx$ metade de $P^\top LU$, e Gauss $\approx P^\top LU$ (mesmo número de operações); em (b), $P^\top LU$ e Cholesky muito mais rápidos que Gauss (razão $\rightarrow 10$ ou mais quando n cresce), e Cholesky ainda à frente.

Medição. Tempo de parede (`time.perf_counter`) só da resolução (a geração das matrizes não entra). Cada tempo total (10 sistemas) foi medido 5 vezes e foi registrada a mediana, para reduzir o ruído do sistema operacional.

2.6 Resultados

n	(a) 10 matrizes distintas (s)			(b) 1 matriz, 10 vetores (s)		
	Gauss	$P^\top LU$	Cholesky	Gauss	$P^\top LU$	Cholesky
100	0.0228	0.0115	0.0078	0.0191	0.0034	0.0031
200	0.0797	0.0296	0.0186	0.0658	0.0074	0.0060
300	0.3348	0.0501	0.0290	0.2586	0.0119	0.0097
400	1.0065	0.0814	0.0498	0.8774	0.0170	0.0141
500	1.8321	0.1111	0.0592	1.8261	0.0227	0.0179

Tabela 3: Tempo total de execução (em segundos, para os 10 sistemas) por método e por n . Os valores absolutos dependem da máquina; o que importa é a relação entre os métodos.

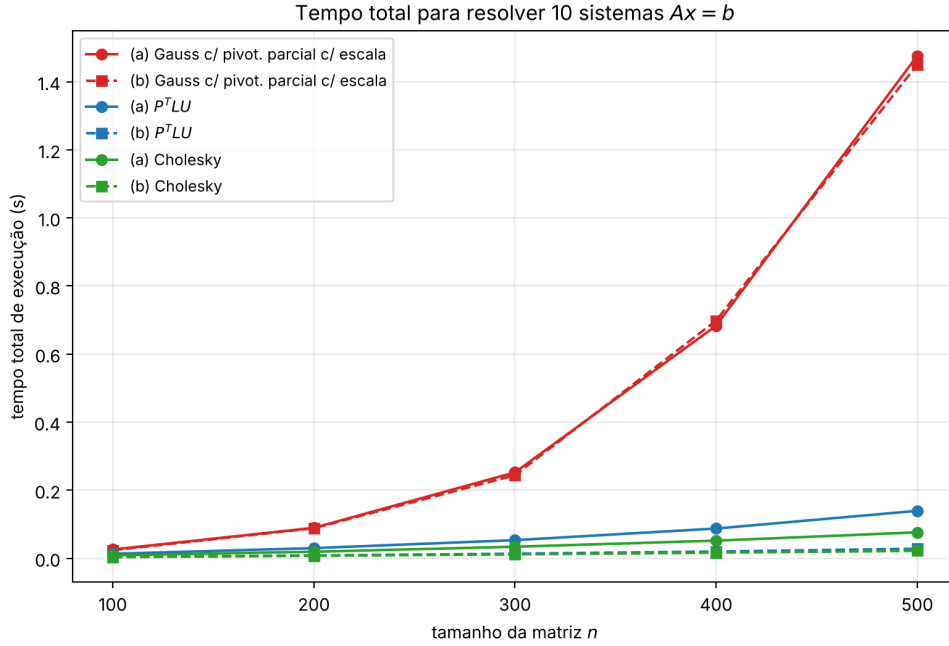


Figura 1: As 6 curvas de tempo $\times n$. Linhas cheias: item (a); tracejadas: item (b).

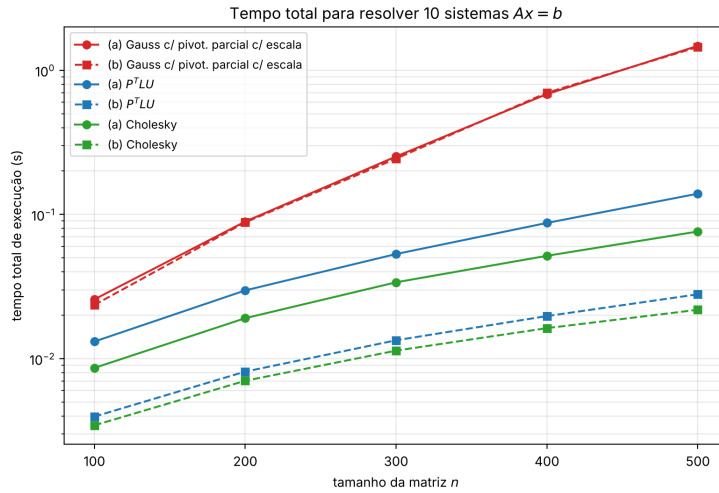


Figura 2: Figura complementar, com os mesmos dados da Figura 1 e eixo vertical logarítmico (apenas para enxergar as curvas pequenas). A figura pedida no enunciado, com as 6 curvas, é a Figura 1.

2.7 Respostas e discussão

Item (a): o menor tempo foi o da decomposição de Cholesky, para todos os valores de n . Em $n = 500$, $P^T LU$ levou 1,88 vez o tempo de Cholesky, perto da razão teórica 2 ($\frac{n^3}{3}$ contra $\frac{n^3}{6}$).

Item (b): o menor tempo também foi o de Cholesky, para todos os n , com $P^T LU$ logo atrás e a eliminação de Gauss muito atrás (≈ 80 vezes mais lenta que $P^T LU$ e ≈ 102 vezes mais lenta que Cholesky em $n = 500$). Aqui aparece o efeito pedido no enunciado: Gauss paga $O(n^3/3)$ para cada b , enquanto as fatorações pagam $O(n^3)$ uma única vez e depois só $O(n^2)$ por vetor. Por isso as curvas (b) de $P^T LU$ e Cholesky ficam bem abaixo das curvas (a), enquanto as duas curvas de Gauss praticamente coincidem (o mesmo trabalho é feito 10 vezes nos dois itens). A diferença entre Cholesky e $P^T LU$ em (b) é menor (1,27 vez) porque as 10 substituições, que custam o mesmo nos dois métodos, passam a pesar bastante no total.

Por que Gauss ficou tão mais lento que $P^\top LU$ em (a), se o número de operações é o mesmo? A contagem de operações ($\approx n^3/3$ para ambos) não é a única coisa que define o tempo: a forma como o algoritmo percorre a memória também conta. Na forma do [Algoritmo 6.3], cada etapa i *reescreve* toda a submatriz restante $(n-i) \times (n-i)$ por uma atualização de posto 1 (no código, `np.outer` cria uma matriz temporária de ordem $n-i$ por etapa, e a subtração lê e grava a submatriz inteira), ou seja, cada entrada é lida e gravada $O(n)$ vezes. Na forma de produto interno do [Algoritmo 6.4] (e do 6.6), cada entrada de L e U é calculada uma única vez como $a_{ij} - \sum_k l_{ik}u_{kj}$, isto é, por produtos escalares sobre fatores já prontos, com muito menos tráfego de memória. É a mesma aritmética em outra ordem, e é exatamente por isso que bibliotecas profissionais reorganizam a eliminação dessa forma. O mesmo efeito explica por que, em (b), a razão medida Gauss/ $P^\top LU$ em $n = 500$ (≈ 80) é bem maior que a razão teórica $\frac{10n^3/3}{n^3/3+10n^2} \approx 9,4$. Além disso, para $n \leq 500$ o tempo das fatorações ainda é dominado pelo custo fixo de cada iteração do laço em Python: a inclinação medida entre $n = 100$ e $n = 500$ no gráfico log-log foi $\approx 2,7$ – $2,8$ para Gauss e apenas $\approx 1,1$ – $1,4$ para $P^\top LU$ e Cholesky (e não 3); para n maiores, a razão n^3 passa a dominar. Nada disso altera as conclusões: *Cholesky foi o mais rápido nos dois itens*, como prevê a contagem de operações, e em (b) as fatorações reaproveitadas vencem a eliminação por larga margem.

3 Questão 3 — Temperatura de equilíbrio numa placa circular

(a) Sistema $t = Mt + b$ e $At = b$

A **propriedade discreta do valor médio** (Anton & Rorres, §10.11) afirma que, no equilíbrio, a temperatura num ponto interior da malha é aproximadamente a *média aritmética* das temperaturas dos quatro pontos vizinhos (acima, abaixo, à esquerda, à direita). Ela é a discretização da equação de Laplace $u_{xx} + u_{yy} = 0$: com a fórmula de diferenças centradas de segunda ordem e espaçamento h ,

$$\frac{u_E - 2u_P + u_W}{h^2} + \frac{u_N - 2u_P + u_S}{h^2} = 0 \iff u_P = \frac{1}{4}(u_N + u_S + u_E + u_W).$$

Numerando t_1 (superior esquerdo), t_2 (superior direito), t_3 (inferior esquerdo) e t_4 (inferior direito), e lembrando que o contorno vale 0 na metade esquerda e 1 na metade direita (figura do enunciado):

$$\begin{aligned} t_1 &= \frac{1}{4} \left(\underbrace{0}_{\text{cima}} + \underbrace{0}_{\text{esq.}} + t_2 + t_3 \right), & t_2 &= \frac{1}{4} \left(\underbrace{1}_{\text{cima}} + \underbrace{1}_{\text{dir.}} + t_1 + t_4 \right), \\ t_3 &= \frac{1}{4} \left(\underbrace{0}_{\text{baixo}} + \underbrace{0}_{\text{esq.}} + t_1 + t_4 \right), & t_4 &= \frac{1}{4} \left(\underbrace{1}_{\text{baixo}} + \underbrace{1}_{\text{dir.}} + t_2 + t_3 \right). \end{aligned}$$

Em forma matricial, $t = Mt + b$ com

$$M = \frac{1}{4} \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ \frac{1}{2} \\ 0 \\ \frac{1}{2} \end{bmatrix}.$$

Como $t = Mt + b \iff (I - M)t = b$, o sistema é $At = b$ com

$$A = I - M = \begin{bmatrix} 1 & -\frac{1}{4} & -\frac{1}{4} & 0 \\ -\frac{1}{4} & 1 & 0 & -\frac{1}{4} \\ -\frac{1}{4} & 0 & 1 & -\frac{1}{4} \\ 0 & -\frac{1}{4} & -\frac{1}{4} & 1 \end{bmatrix}.$$

(No código, M e b são montados automaticamente a partir da lista de vizinhos de cada ponto.) Observe que A é simétrica, tem diagonal positiva e $1 > \frac{1}{4} + \frac{1}{4} = R_i$: pelo teorema da Questão 1, já *esperamos* que A seja PD, e o item (b) confirma isso algoritmicamente.

(b) Cholesky como teste de positividade

Proposição 1. *Seja A simétrica. O Algoritmo 6.6 termina com todos os radicandos $d_i = a_{ii} - \sum_{k < i} l_{ik}^2$ estritamente positivos se e somente se A é positiva definida.*

Demonstração. ($\Leftarrow$) Se A é PD, cada submatriz principal dominante A_k é PD e $\det A_k > 0$ [Teorema 6.25, p. 462]. Por indução, se o algoritmo chegou à etapa k , então $A_k = L_k L_k^\top$ e $\det A_k = l_{11}^2 \cdots l_{k-1,k-1}^2 d_k$, de modo que $d_k = \det A_k / (l_{11} \cdots l_{k-1,k-1})^2 > 0$. ($\Rightarrow$) Se todos os $d_i > 0$, obtemos $A = LL^\top$ com $l_{ii} = \sqrt{d_i} > 0$; $L^\top$ é invertível e, para $x \neq 0$, $x^\top A x = x^\top L L^\top x = \|L^\top x\|_2^2 > 0$. (É o conteúdo do [Teorema 6.26 e Corolário 6.28, p. 463].) $\square$

Adaptação feita (`cholesky_com_teste`, que reaproveita as fórmulas de `fatoracao_cholesky` da Questão 2): (1) verifica-se a simetria (exigida pela Definição 6.22); (2) executa-se o Algoritmo 6.6 e, se algum radicando d_i não for estritamente positivo, devolve-se “não é PD” em vez de calcular a raiz de um número não-positivo. Em ponto flutuante, “ $d_i > 0$ ” é testado com tolerância relativa ($d_i \leq 10^{-10} a_{ii}$ é tratado como zero): sem ela, uma matriz apenas semidefinida (singular) pode gerar $d_i \approx 10^{-17} > 0$ por erro de arredondamento e ser aceita como PD (ver a ilustração numérica da Questão 4). O custo do teste é o da própria fatoração. Como verificação, o detector responde corretamente “não PD” para $\begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$ (autovalores 3 e -1): $d_2 = 1 - 4 = -3 < 0$. Para exercitar o caminho alternativo, o sistema $\begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix} x = (3, 3)^\top$ é então resolvido por $P^\top LU$ e devolve $x = (1, 1)^\top$, como esperado.

Aplicação a $A = I - M$. O teste retornou **A positiva definida**; portanto o sistema foi resolvido por Cholesky. Os fatores (exatos e numéricos) são

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -\frac{1}{4} & \frac{\sqrt{15}}{4} & 0 & 0 \\ -\frac{1}{4} & -\frac{1}{4\sqrt{15}} & \sqrt{\frac{14}{15}} & 0 \\ 0 & -\frac{1}{\sqrt{15}} & -\frac{4}{\sqrt{210}} & \sqrt{\frac{6}{7}} \end{bmatrix} \approx \begin{bmatrix} 1 & 0 & 0 & 0 \\ -0.25 & 0.9682 & 0 & 0 \\ -0.25 & -0.0645 & 0.9661 & 0 \\ 0 & -0.2582 & -0.2760 & 0.9258 \end{bmatrix},$$

com $d = (1, \frac{15}{16}, \frac{14}{15}, \frac{6}{7})$, todos positivos, e $\|LL^\top - A\|_{\max} \approx 2 \times 10^{-16}$. Por exemplo, $l_{22} = \sqrt{1 - (-\frac{1}{4})^2} = \frac{\sqrt{15}}{4}$ e $l_{32} = (0 - (-\frac{1}{4})(-\frac{1}{4})) / l_{22} = -\frac{1}{4\sqrt{15}}$. Substituição progressiva $Ly = b$: $y \approx (0, 0.5164, 0.0345, 0.6944)^\top$; regressiva $L^\top t = y$:

$$\boxed{t_1 = 0.25, \quad t_2 = 0.75, \quad t_3 = 0.25, \quad t_4 = 0.75}$$

e $\|t - (Mt + b)\|_\infty \approx 2 \times 10^{-16}$.

Conferência analítica. A placa e o contorno são simétricos em relação ao eixo horizontal, logo $t_1 = t_3$ e $t_2 = t_4$. A primeira equação dá $4t_1 = t_2 + t_1$, isto é, $t_2 = 3t_1$; a segunda dá $4t_2 = 2 + t_1 + t_2$, isto é, $3t_2 = 2 + t_1$. Então $9t_1 = 2 + t_1$, logo $t_1 = \frac{1}{4}$ e $t_2 = \frac{3}{4}$, exatamente o resultado numérico. Note ainda que $t_1 + t_2 = 1$: trocar esquerda $\leftrightarrow$ direita troca $T \mapsto 1 - T$, como esperado fisicamente.

(c) Figura da placa

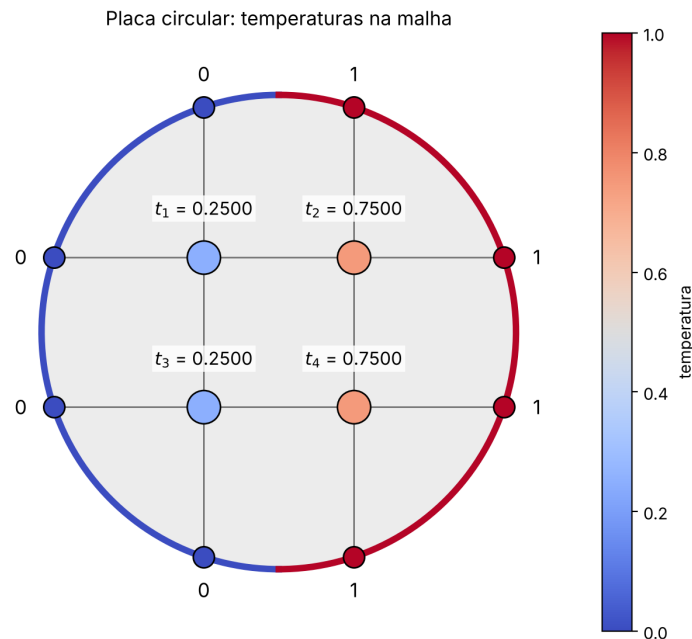


Figura 3: Malha sobre a placa: 8 pontos de contorno (0 à esquerda, 1 à direita) e 4 pontos interiores coloridos pela temperatura calculada (escala *coolwarm*). Os pontos de contorno foram colocados à mesma distância h dos interiores, de modo que a malha seja uniforme.

4 Questão 4 — Matriz de Gram $A = B^T B$

Seja $B \in \mathbb{R}^{m \times n}$ e $A = B^\top B \in \mathbb{R}^{n \times n}$. Note que $a_{ij} = \sum_{k=1}^m b_{ki} b_{kj} = \langle B_{:,i}, B_{:,j} \rangle$ é o produto escalar entre as colunas i e j de B .

(a) A é simétrica

Usando $(XY)^{\top} = Y^{\top} X^{\top}$ e $(X^{\top})^{\top} = X$:

$$A^\top = (B^\top B)^\top = B^\top (B^\top)^\top = B^\top B = A.$$

(Em coordenadas: $a_{ij} = \langle B_{:,i}, B_{:,j} \rangle = \langle B_{:,j}, B_{:,i} \rangle = a_{ji}$, pois o produto escalar é comutativo.)

(b) A é positiva semidefinida

Para qualquer $x \in \mathbb{R}^n$, seja $y = Bx \in \mathbb{R}^m$. Então, pela associatividade do produto de matrizes e pelo item (a),

$$x^\top Ax = x^\top B^\top Bx = (Bx)^\top (Bx) = y^\top y = \sum_{k=1}^m y_k^2 = \|Bx\|_2^2 \geq 0,$$

pois é uma soma de quadrados de números reais. Logo A é positiva semidefinida.

Quando A é de fato positiva definida? Pela conta acima, $x^\top Ax = 0 \iff Bx = 0$. Assim, A é PD (no sentido da [Definição 6.22]) se e somente se $Bx = 0$ só admite $x = 0$, isto é, se as colunas de B são linearmente independentes (posto n , o que exige $m \geq n$). Se $m < n$, há $z \neq 0$ com $Bz = 0$ e então $z^\top Az = 0$: A é apenas semidefinida. Esse é o caso das *equações normais* $B^\top Bx = B^\top y$ dos mínimos quadrados [Seção 8.1]; não por acaso, Cholesky criou sua fatoração justamente para resolver problemas de mínimos quadrados [nota histórica, p. 464].

Ilustração numérica (questao4.py). Para B aleatórias 5×3 , 3×3 e 2×4 , obteve-se $\max |A - A^\top| = 0$ e $x^\top A x = \|Bx\|_2^2 \geq 0$ em 20 000 vetores sorteados. O teste de Cholesky da Questão 3 classificou A como PD nos casos 5×3 e 3×3 (colunas independentes) e como *não* PD no caso 2×4 , em que o programa encontrou $z \neq 0$ com $Bz = 0$ e $z^\top A z \approx 10^{-17}$ (zero, a menos de arredondamento). Para não depender de um único sorteio, o script repete o teste em 5 000 matrizes B com $m < n$ (A singular, apenas semidefinida): o teste aceitou A como PD em apenas 2 casos (0,04%), de matrizes numericamente quase singulares, em que o arredondamento impede distinguir PD de semidefinida; sem a tolerância relativa seriam 693 casos (13,9%). Em outros 5 000 sorteios com $m \geq n$ (colunas independentes), o teste respondeu PD em todos.

A Códigos

Execução dos scripts (na mesma pasta, pois `questao2.py` importa `questao1.py`, `questao3.py` importa `questao2.py` e `questao4.py` importa `questao3.py`): `python questao1.py`, `python questao2.py` (gera `q2_tempos.png`, `q2_tempos_log.png` e `q2_resultados.json`), `python questao3.py` (gera `q3_placa.png`) e `python questao4.py`.

questao1.py

```

1  """
2  Questão 1 - Gerador aleatório de matrizes positivas definidas.
3
4  Ideia (Teorema do enunciado + Burden, Def. 6.20 e Def. 6.22):
5      Se A é simétrica, estritamente diagonal dominante e tem diagonal
6      não-negativa, então A é positiva definida.
7
8  Algoritmo:
9      1. Sorteia os elementos abaixo da diagonal a_ij ~ U(-1, 1), i > j.
10     2. Espelha: a_ji = a_ij (garante A = A^T).
11     3. Para cada linha i, define
12         a_ii = sum_{j != i} |a_ij| + delta_i, delta_i ~ U(1, 2) (> 0)
13         o que garante |a_ii| > sum_{j != i} |a_ij| (dominância estrita)
14         e a_ii > 0 (diagonal positiva).
15
16  Só usamos o NumPy como "contêiner" de vetores/matrizes e para sortear números.
17  Nenhuma rotina de álgebra linear pronta (numpy.linalg, scipy) é usada.
18  """
19
20  import numpy as np
21
22
23  def gerar_matriz_pd(n, rng=None, faixa=1.0, margem=(1.0, 2.0)):
24      """Gera uma matriz n x n simétrica, estritamente diagonal dominante,
25      com diagonal positiva (logo, positiva definida)."""
26      if rng is None:
27          rng = np.random.default_rng()
28      A = np.zeros((n, n))
29      # 1) parte estritamente inferior aleatória
30      for i in range(1, n):
31          A[i, :i] = rng.uniform(-faixa, faixa, size=i)
32      # 2) simetria: A = L + L^T
33      A = A + A.T
34      # 3) diagonal: soma dos módulos fora da diagonal + folga positiva
35      soma_fora = np.abs(A).sum(axis=1) # diagonal ainda é zero aqui
36      delta = rng.uniform(margem[0], margem[1], size=n)
37      for i in range(n):
38          A[i, i] = soma_fora[i] + delta[i]
39      return A
40
41
42  # ----- verificações
43  def eh_simetrica(A, tol=0.0):
44      n = A.shape[0]
45      for i in range(n):
46          for j in range(i + 1, n):
47              if abs(A[i, j] - A[j, i]) > tol:
48                  return False

```

```

49     return True
50
51
52 def eh_estritamente_diagonal_dominante(A):
53     n = A.shape[0]
54     for i in range(n):
55         soma = 0.0
56         for j in range(n):
57             if j != i:
58                 soma += abs(A[i, j])
59             if not abs(A[i, i]) > soma:
60                 return False
61     return True
62
63
64 def diagonal_nao_negativa(A):
65     return all(A[i, i] >= 0 for i in range(A.shape[0]))
66
67
68 def menor_folga(A):
69     """min_i ( a_ii - sum_{j!=i} |a_ij| ): quanto maior que zero, mais 'folgada'
70     é a dominância diagonal. Pela demonstração,  $x^T A x \geq \text{menor\_folga} * ||x||^2$ ."""
71     n = A.shape[0]
72     return min(A[i, i] - (np.abs(A[i, :]).sum() - abs(A[i, i])) for i in range(n))
73
74
75 def teste_empirico_forma_quadratica(A, amostras=20000, rng=None):
76     """Sorteia vetores  $x \neq 0$  e devolve o menor valor de  $x^T A x / x^T x$ ."""
77     if rng is None:
78         rng = np.random.default_rng(0)
79     n = A.shape[0]
80     menor = np.inf
81     for _ in range(amostras):
82         x = rng.normal(size=n)
83         q = x @ (A @ x)
84         menor = min(menor, q / (x @ x))
85     return menor
86
87
88 if __name__ == "__main__":
89     rng = np.random.default_rng(2026)
90     np.set_printoptions(precision=4, suppress=True, linewidth=120)
91
92     for k, n in enumerate([3, 4, 5], start=1):
93         A = gerar_matriz_pd(n, rng)
94         print(f"===== Exemplo {k}: n = {n} =====")
95         print(A)
96         print(f" simétrica? {eh_simetrica(A)}")
97         print(f" estritamente diag. dominante? {eh_estritamente_diagonal_dominante(A)}")
98         print(f" diagonal não-negativa? {diagonal_nao_negativa(A)}")
99         print(f" folga mínima min_i(a_ii - R_i) = {menor_folga(A):.4f}")
100        print(f" min  $x^T A x / x^T x$  (20000 x aleatórios) = "
101              f"{teste_empirico_forma_quadratica(A, rng=rng):.4f} (> 0)")
102        print()

```

questao2.py

```

1  """
2  Questão 2 - Eliminação Gaussiana com pivoteamento parcial com escala,
3  Decomposição  $P^T LU$ , Decomposição de Cholesky e substituições
4  (progressiva / regressiva) para resolver  $Ax = b$ ; experimento de tempos.
5
6  Referências (Burden, Faires & Burden, Análise Numérica, 10ª ed.):
7      * Algoritmo 6.1 (p. 403) - eliminação de Gauss + substituição regressiva
8      * Algoritmo 6.3 (p. 417) - pivoteamento parcial com escala
9      * Teorema 6.19 / Alg. 6.4 (pp. 448-450) e p. 452 - fatoração LU e  $A = P^T L U$ 
10     * Corolário 6.28 / Algoritmo 6.6 (pp. 463-464) - fatoração de Cholesky  $A = L L^T$ 
11
12  Restrição da lista: "sem bibliotecas numéricas avançadas". O NumPy é usado
13  apenas como contêiner de vetores/matrizes e para operações elementares
14  (somas, produtos escalares, operações de linha). NÃO se usa numpy.linalg

```

```

15 nem scipy: todos os algoritmos estão escritos abaixo.
16
17 Uso:
18     python questao2.py # roda o experimento completo (itens a, b, c)
19 """
20
21 import json
22 import time
23
24 import numpy as np
25
26 from questao1 import gerar_matriz_pd
27
28 class MatrizSingular(Exception):
29     pass
30
31
32 class NaoPositivaDefinida(Exception):
33     pass
34
35
36
37 # =====
38 # Substituições (as "variações de substituição reversa")
39 # =====
40 def substituicao_progressiva(L, b, diagonal_unitaria=False):
41     """Resolve  $Ly = b$ ,  $L$  triangular inferior (forward substitution).
42      $y_1 = b_1 / l_{11}$ ;  $y_i = (b_i - \sum_{j<i} l_{ij} y_j) / l_{ii}$ ."""
43     n = L.shape[0]
44     y = np.zeros(n)
45     for i in range(n):
46         s = b[i] - L[i, :i] @ y[:i]
47         y[i] = s if diagonal_unitaria else s / L[i, i]
48     return y
49
50
51 def substituicao_regressiva(U, y):
52     """Resolve  $Ux = y$ ,  $U$  triangular superior (back substitution, Alg. 6.1 passos 8-9).
53      $x_n = y_n / u_{nn}$ ;  $x_i = (y_i - \sum_{j>i} u_{ij} x_j) / u_{ii}$ ."""
54     n = U.shape[0]
55     x = np.zeros(n)
56     for i in range(n - 1, -1, -1):
57         if U[i, i] == 0:
58             raise MatrizSingular("pivô nulo na substituição regressiva")
59         x[i] = (y[i] - U[i, i + 1:] @ x[i + 1:]) / U[i, i]
60     return x
61
62
63 def substituicao_regressiva_transposta(L, y):
64     """Resolve  $L^T x = y$  usando apenas  $L$  (sem montar  $L^T$ ):
65      $x_i = (y_i - \sum_{j>i} l_{ji} x_j) / l_{ii}$ . (passos 9-10 após o Alg. 6.6)"""
66     n = L.shape[0]
67     x = np.zeros(n)
68     for i in range(n - 1, -1, -1):
69         x[i] = (y[i] - L[i + 1:, i] @ x[i + 1:]) / L[i, i]
70     return x
71
72
73 # =====
74 # 1) Eliminação Gaussiana com pivoteamento parcial com escala (Alg. 6.3)
75 # =====
76 def fatores_de_escalas(A):
77     """ $s_i = \max_j |a_{ij}|$  (Passo 1 do Alg. 6.3)."""
78     s = np.abs(A).max(axis=1)
79     if np.any(s == 0):
80         raise MatrizSingular("linha nula: não existe solução única")
81     return s
82
83
84 def eliminacao_gauss_escalas(A, b):
85     """Resolve  $Ax = b$  por eliminação de Gauss com pivoteamento parcial com escala.
86     As trocas de linha são 'simuladas' com o vetor NLINHA, como no livro."""
87     a = np.array(A, dtype=float) # cópia: não altera A

```

```

88     bb = np.array(b, dtype=float)
89     n = a.shape[0]
90     s = fatores_de_escalas(a)
91     nlinha = np.arange(n) # NLINHA(i) = i
92
93     for i in range(n - 1):
94         # Passo 3: menor p >= i que maximiza |a(NLINHA(p), i)| / s(NLINHA(p))
95         candidatas = nlinha[i:]
96         razoes = np.abs(a[candidatas, i]) / s[candidatas]
97         p = i + int(np.argmax(razoes))
98         if a[nlinha[p], i] == 0:
99             raise MatrizSingular("não existe solução única")
100         # Passo 5: troca simulada
101         if p != i:
102             nlinha[i], nlinha[p] = nlinha[p], nlinha[i]
103         piv = nlinha[i]
104         abaixo = nlinha[i + 1:]
105         # Passos 6-7: m_ji = a_ji / a_ii ; E_j <- E_j - m_ji E_i
106         m = a[abaixo, i] / a[piv, i]
107         a[abaixo, i:] -= np.outer(m, a[piv, i:])
108         bb[abaixo] -= m * bb[piv]
109
110     if a[nlinha[n - 1], n - 1] == 0:
111         raise MatrizSingular("não existe solução única")
112     # Substituição regressiva na ordem NLINHA (sistema triangular superior)
113     return substituicao_regressiva(a[nlinha], bb[nlinha])
114
115 # =====
116 # 2) Decomposição P^T L U (PA = LU <=> A = P^T L U)
117 # =====
118 def fatoracao_ptlu(A, com_escalas=True):
119     """Fatoração de Doolittle (Alg. 6.4, l_ii = 1) com trocas de linhas.
120
121     Usa as fórmulas do Alg. 6.4 em forma de produto interno:
122         u_ij = a_ij - sum_{k<i} l_ik u_kj (linha i de U)
123         l_ji = (a_ji - sum_{k<i} l_jk u_ki) / u_ii (coluna i de L)
124     Antes de dividir por u_ii, escolhe-se o pivô na coluna i pelo critério do
125     pivoteamento parcial com escalas (Alg. 6.3) e trocam-se as linhas.
126
127     Devolve (LU, perm): L (diagonal unitária, abaixo da diagonal) e U (diagonal
128     e acima) guardadas na mesma matriz, e o vetor de permutação perm tal que
129     P[i, perm[i]] = 1, ou seja, (PA)[i] = A[perm[i]] e PA = LU."""
130     a = np.array(A, dtype=float)
131     n = a.shape[0]
132     perm = np.arange(n)
133     s = fatores_de_escalas(a) if com_escalas else np.ones(n)
134
135     for i in range(n):
136         # candidatos a pivô: a_ji - sum_{k<i} l_jk u_ki , j = i..n-1
137         a[i:, i] -= a[i:, :i] @ a[:i, i]
138         p = i + int(np.argmax(np.abs(a[i:, i]) / s[i:]))
139         if a[p, i] == 0:
140             raise MatrizSingular("fatoração impossível: matriz singular")
141         if p != i: # troca de linhas (inclui parte já calculada de L)
142             a[[i, p]] = a[[p, i]]
143             perm[[i, p]] = perm[[p, i]]
144             s[[i, p]] = s[[p, i]]
145         # linha i de U (colunas i+1..n-1)
146         a[i, i + 1:] -= a[i, :i] @ a[:i, i + 1:]
147         # coluna i de L (multiplicadores)
148         a[i + 1:, i] /= a[i, i]
149     return a, perm
150
151
152 def resolver_ptlu(LU, perm, b):
153     """Ax = b com A = P^T L U: L y = P b (progressiva), U x = y (regressiva)."""
154     pb = np.asarray(b, dtype=float)[perm]
155     y = substituicao_progressiva(LU, pb, diagonal_unitaria=True)
156     return substituicao_regressiva(LU, y)
157
158
159 def extrair_P_L_U(LU, perm):

```

```

161     n = LU.shape[0]
162     L = np.tril(LU, -1) + np.eye(n)
163     U = np.triu(LU)
164     P = np.zeros((n, n))
165     P[np.arange(n), perm] = 1.0
166     return P, L, U
167
168
169 # =====
170 # 3) Decomposição de Cholesky  $A = L L^T$  (Alg. 6.6)
171 # =====
172 def fatoracao_cholesky(A):
173     """ $l_{ii} = \sqrt{a_{ii} - \sum_{k<i} l_{ik}^2}$ 
174      $l_{ji} = (a_{ji} - \sum_{k<i} l_{jk} l_{ik}) / l_{ii}$ ,  $j > i$ ."""
175     A = np.asarray(A, dtype=float)
176     n = A.shape[0]
177     L = np.zeros((n, n))
178     for i in range(n):
179         d = A[i, i] - L[i, :i] @ L[i, :i]
180         if d <= 0:
181             raise NaoPositivaDefinida(f"pivô {d:.3e} <= 0 na etapa {i + 1}")
182         L[i, i] = np.sqrt(d)
183         L[i + 1:, i] = (A[i + 1:, i] - L[i + 1:, :i] @ L[i, :i]) / L[i, i]
184     return L
185
186
187 def resolver_cholesky(L, b):
188     """ $Ax = b$  com  $A = L L^T$ :  $L y = b$  (progressiva),  $L^T x = y$  (regressiva)."""
189     y = substituicao_progressiva(L, b)
190     return substituicao_regressiva_transposta(L, y)
191
192
193 # =====
194 # Experimento
195 # =====
196 def vetor_b_nao_nulo(n, rng):
197     while True:
198         b = rng.uniform(-10, 10, size=n)
199         if np.any(b != 0):
200             return b
201
202
203 def residuo(A, x, b):
204     return float(np.max(np.abs(A @ x - b)))
205
206
207 def _mediana(v):
208     v = sorted(v)
209     return v[len(v) // 2]
210
211
212 def item_a(ns, reps, rng, repeticoes=5):
213     """10 matrizes PD distintas e 10 vetores b: um sistema  $A_k x = b_k$  por par.
214     Cada  $A_k$  é diferente, então os três métodos têm de processar  $A_k$  do zero.
215     O tempo total (10 sistemas) é medido 'repeticoes' vezes e guarda-se a mediana,
216     para reduzir o ruído do sistema operacional."""
217     tempos = {"Gauss": [], "PtLU": [], "Cholesky": []}
218     for n in ns:
219         As = [gerar_matriz_pd(n, rng) for _ in range(reps)]
220         bs = [vetor_b_nao_nulo(n, rng) for _ in range(reps)]
221         medidas = {"Gauss": [], "PtLU": [], "Cholesky": []}
222         res_max = 0.0
223         for _ in range(repeticoes):
224             t0 = time.perf_counter()
225             xs1 = [eliminacao_gauss_escalas(A, b) for A, b in zip(As, bs)]
226             t1 = time.perf_counter()
227             xs2 = []
228             for A, b in zip(As, bs):
229                 LU, perm = fatoracao_ptlu(A) # cada A é nova: precisa fatorar
230                 xs2.append(resolver_ptlu(LU, perm, b))
231             t2 = time.perf_counter()
232             xs3 = []
233             for A, b in zip(As, bs):

```

```

234         L = fatoracao_cholesky(A)
235         xs3.append(resolver_cholesky(L, b))
236         t3 = time.perf_counter()
237         medidas["Gauss"].append(t1 - t0)
238         medidas["PtLU"].append(t2 - t1)
239         medidas["Cholesky"].append(t3 - t2)
240     for A, b, x1, x2, x3 in zip(As, bs, xs1, xs2, xs3):
241         res_max = max(res_max, residuo(A, x1, b), residuo(A, x2, b), residuo(A, x3,
242             b))
243     t = {k: _mediana(v) for k, v in medidas.items()}
244     for k in tempos:
245         tempos[k].append(t[k])
246     print(f"(a) n={n:4d} Gauss={t['Gauss']:.4f}s PtLU={t['PtLU']:.4f}s "
247           f"Cholesky={t['Cholesky']:.4f}s max||Ax-b||_inf={res_max:.2e}")
248     return tempos
249
250 def item_b(ns, reps, rng, repeticoes=5):
251     """1 matriz PD e 10 vetores b. Gauss refaz a eliminação a cada b;
252     P~T LU e Cholesky fatoram UMA vez e reaproveitam a fatoração
253     (só as substituições,  $O(n^2)$ ), são repetidas para cada b)."""
254     tempos = {"Gauss": [], "PtLU": [], "Cholesky": []}
255     for n in ns:
256         A = gerar_matriz_pd(n, rng)
257         bs = [vetor_b_nao_nulo(n, rng) for _ in range(reps)]
258         medidas = {"Gauss": [], "PtLU": [], "Cholesky": []}
259         for _ in range(repeticoes):
260             t0 = time.perf_counter()
261             xs1 = [eliminacao_gauss_escala(A, b) for b in bs]
262             t1 = time.perf_counter()
263
264             LU, perm = fatoracao_ptlu(A) # fatora uma única vez
265             xs2 = [resolver_ptlu(LU, perm, b) for b in bs]
266             t2 = time.perf_counter()
267
268             L = fatoracao_cholesky(A) # fatora uma única vez
269             xs3 = [resolver_cholesky(L, b) for b in bs]
270             t3 = time.perf_counter()
271             medidas["Gauss"].append(t1 - t0)
272             medidas["PtLU"].append(t2 - t1)
273             medidas["Cholesky"].append(t3 - t2)
274
275         res_max = 0.0
276         for b, x1, x2, x3 in zip(bs, xs1, xs2, xs3):
277             res_max = max(res_max, residuo(A, x1, b), residuo(A, x2, b), residuo(A, x3,
278                 b))
279         t = {k: _mediana(v) for k, v in medidas.items()}
280         for k in tempos:
281             tempos[k].append(t[k])
282         print(f"(b) n={n:4d} Gauss={t['Gauss']:.4f}s PtLU={t['PtLU']:.4f}s "
283               f"Cholesky={t['Cholesky']:.4f}s max||Ax-b||_inf={res_max:.2e}")
284     return tempos
285
286 def plotar(ns, ta, tb, arquivo="q2_tempos.png", log=False):
287     import matplotlib
288     matplotlib.use("Agg")
289     import matplotlib.pyplot as plt
290
291     cores = {"Gauss": "tab:red", "PtLU": "tab:blue", "Cholesky": "tab:green"}
292     rotulo = {"Gauss": "Gauss c/ pivot. parcial c/ escala",
293              "PtLU": r"$P^T LU$", "Cholesky": "Cholesky"}
294     fig, ax = plt.subplots(figsize=(8, 5.5))
295     for k in ["Gauss", "PtLU", "Cholesky"]:
296         ax.plot(ns, ta[k], "-o", color=cores[k], label=f"(a) {rotulo[k]}")
297         ax.plot(ns, tb[k], "--s", color=cores[k], label=f"(b) {rotulo[k]}")
298     ax.set_xlabel("tamanho da matriz $n$")
299     ax.set_ylabel("tempo total de execução (s)")
300     ax.set_title("Tempo total para resolver 10 sistemas $Ax=b$")
301     ax.set_xticks(ns)
302     if log:
303         ax.set_yscale("log")
304     ax.grid(alpha=0.3, which="both")

```

```

305     ax.legend(fontsize=9)
306     fig.tight_layout()
307     fig.savefig(arquivo, dpi=200)
308     plt.close(fig)
309
310
311 def demonstracao_pequena():
312     """Mostra os três métodos num sistema 4x4 e confere que  $A = P^T L U$  e  $A = L L^T$ ."""
313     rng = np.random.default_rng(1)
314     A = gerar_matriz_pd(4, rng)
315     b = vetor_b_nao_nulo(4, rng)
316     np.set_printoptions(precision=5, suppress=True)
317     print("A =\n", A, "\nb =", b)
318     print("Gauss (escala):", eliminacao_gauss_escala(A, b))
319     LU, perm = fatoracao_ptlu(A)
320     P, L, U = extrair_P_L_U(LU, perm)
321     print("P^T LU :", resolver_ptlu(LU, perm, b),
322           " ||P^T L U - A||_max =", np.abs(P.T @ L @ U - A).max())
323     Lc = fatoracao_cholesky(A)
324     print("Cholesky :", resolver_cholesky(Lc, b),
325           " ||L L^T - A||_max =", np.abs(Lc @ Lc.T - A).max())
326     print()
327
328
329 if __name__ == "__main__":
330     demonstracao_pequena()
331
332     ns = [100, 200, 300, 400, 500]
333     reps = 10
334     rng = np.random.default_rng(12345)
335
336     # aquecimento (evita que a 1ª medição inclua custos de inicialização)
337     _A = gerar_matriz_pd(50, rng); _b = vetor_b_nao_nulo(50, rng)
338     eliminacao_gauss_escala(_A, _b); resolver_ptlu(*fatoracao_ptlu(_A), _b)
339     resolver_cholesky(fatoracao_cholesky(_A), _b)
340
341     ta = item_a(ns, reps, rng)
342     print()
343     tb = item_b(ns, reps, rng)
344
345     with open("q2_resultados.json", "w") as f:
346         json.dump({"n": ns, "a": ta, "b": tb}, f, indent=2)
347
348     plotar(ns, ta, tb, "q2_tempos.png")
349     plotar(ns, ta, tb, "q2_tempos_log.png", log=True)
350
351     for item, t in (("a", ta), ("b", tb)):
352         totais = {k: sum(v) for k, v in t.items()}
353         venc = [min(t, key=lambda k: t[k][i]) for i in range(len(ns))]
354         print(f"\nItem ({item}) - método mais rápido em cada n: "
355               + ", ".join(f"n={n}: {m}" for n, m in zip(ns, venc)))
356         print(f"tempos somados: " + ", ".join(f"{k}={v:.3f}s" for k, v in totais.items()
357               ()))

```

questao3.py

```

1  """
2  Questão 3 - Temperatura de equilíbrio numa placa circular (Anton & Rorres, §10.11).
3
4  Malha: 4 pontos interiores t1 (sup. esq.), t2 (sup. dir.), t3 (inf. esq.),
5  t4 (inf. dir.) e 8 pontos de contorno: temperatura 0 na metade esquerda da
6  circunferência e 1 na metade direita.
7
8  Propriedade discreta do valor médio: a temperatura num ponto interior é a média
9  das temperaturas nos 4 vizinhos (cima, baixo, esquerda, direita).
10 Isso dá  $t = M t + b \rightarrow (I - M) t = b \rightarrow A t = b$ .
11
12 Item (b): Cholesky "adaptado" - além de fatorar, decide se A é positiva definida
13 (Burden, Teorema 6.26 / Corolário 6.28: A simétrica é PD  $\Leftrightarrow$  a fatoração
14  $A = L L^T$  existe com  $l_{ii} > 0$ , i.e., todos os radicandos são  $> 0$ ).
15 """

```

```

16
17 import numpy as np
18
19 from questao2 import (resolver_ptlu, fatoracao_ptlu, substituicao_progressiva,
20                       substituicao_regressiva_transposta)
21
22
23 # ----- (a)
24 def montar_sistema():
25     """Monta M e b a partir da vizinhança de cada ponto interior.
26     Vizinhos do tipo ('t', k) são incógnitas; ('c', valor) são pontos de contorno."""
27     vizinhos = {
28         # ponto: cima baixo esquerda direita
29         0: [("c", 0.0), ("t", 2), ("c", 0.0), ("t", 1)], # t1
30         1: [("c", 1.0), ("t", 3), ("t", 0), ("c", 1.0)], # t2
31         2: [("t", 0), ("c", 0.0), ("c", 0.0), ("t", 3)], # t3
32         3: [("t", 1), ("c", 1.0), ("t", 2), ("c", 1.0)], # t4
33     }
34     M = np.zeros((4, 4))
35     b = np.zeros(4)
36     for i, viz in vizinhos.items():
37         for tipo, v in viz:
38             if tipo == "t":
39                 M[i, v] += 0.25
40             else:
41                 b[i] += 0.25 * v
42     return M, b
43
44 # ----- (b)
45 def cholesky_com_teste(A, tol_sim=1e-12, tol_pd=1e-10):
46     """Algoritmo 6.6 (Cholesky) adaptado.
47     Devolve (eh_pd, L). eh_pd = False se A não for simétrica ou se algum
48     radicando d_i = a_ii - sum_k l_ik^2 não for estritamente positivo
49     (nesse caso L = None).
50
51     Em ponto flutuante, "d_i > 0" é testado com tolerância relativa: d_i <= tol_pd *
52     a_ii
53     é tratado como zero. Sem isso, uma matriz apenas semidefinida (singular) pode
54     produzir d_i ~ 1e-17 > 0 por erro de arredondamento e ser aceita como PD."""
55     A = np.asarray(A, dtype=float)
56     n = A.shape[0]
57     # 1) PD (no sentido do Burden, Def. 6.22) exige simetria
58     for i in range(n):
59         for j in range(i + 1, n):
60             if abs(A[i, j] - A[j, i]) > tol_sim:
61                 return False, None
62     # 2) fatoração; falha <=> A não é PD
63     L = np.zeros((n, n))
64     for i in range(n):
65         d = A[i, i] - L[i, :i] @ L[i, :i]
66         if d <= tol_pd * abs(A[i, i]):
67             return False, None
68         L[i, i] = np.sqrt(d)
69         L[i + 1:, i] = (A[i + 1:, i] - L[i + 1:, :i] @ L[i, :i]) / L[i, i]
70     return True, L
71
72
73 def resolver(A, b):
74     eh_pd, L = cholesky_com_teste(A)
75     if eh_pd:
76         y = substituicao_progressiva(L, b)
77         t = substituicao_regressiva_transposta(L, y)
78         return t, "Cholesky", L
79     LU, perm = fatoracao_ptlu(A)
80     return resolver_ptlu(LU, perm, b), "P^T LU", None
81
82 # ----- (c)
83 def desenhar(t, arquivo="q3_placa.png"):
84     import matplotlib
85     matplotlib.use("Agg")
86     import matplotlib.pyplot as plt

```



```

88 from matplotlib.patches import Circle
89
90 h = 1.0 # espaçamento da malha
91 c = h / 2 # pontos interiores em ( $\pm h/2$ ,  $\pm h/2$ )
92 yb = c + h # contorno a uma distância h dos interiores
93 R = np.hypot(c, yb) # raio do disco que passa pelos 8 pontos
94
95 interiores = {"$t_1$": (-c, c), "$t_2$": (c, c), "$t_3$": (-c, -c), "$t_4$": (c, -c)
96             }
97 contorno = [(-c, yb, 0), (c, yb, 1), (-yb, c, 0), (yb, c, 1),
98             (-yb, -c, 0), (yb, -c, 1), (-c, -yb, 0), (c, -yb, 1)]
99
100 cmap = plt.get_cmap("coolwarm")
101 fig, ax = plt.subplots(figsize=(6.4, 6))
102 ax.add_patch(Circle((0, 0), R, facecolor="#ecec", edgecolor="0.25", lw=1.5, zorder
103               =0))
104 # metades do contorno com a cor da temperatura
105 th = np.linspace(np.pi / 2, 3 * np.pi / 2, 200)
106 ax.plot(R * np.cos(th), R * np.sin(th), color=cmap(0.0), lw=4, zorder=1)
107 ax.plot(-R * np.cos(th), R * np.sin(th), color=cmap(1.0), lw=4, zorder=1)
108 # linhas da malha
109 for v in (-c, c):
110     ax.plot([v, v], [-yb, yb], color="0.45", lw=1, zorder=1)
111     ax.plot([-yb, yb], [v, v], color="0.45", lw=1, zorder=1)
112
113 for x, y, T in contorno:
114     ax.scatter(x, y, s=170, c=[T], cmap=cmap, vmin=0, vmax=1,
115               edgecolors="k", zorder=3)
116     dx, dy = 0.22 * np.sign(x) * (abs(x) > c), 0.22 * np.sign(y) * (abs(y) > c)
117     ax.text(x + dx, y + dy, f"{T:g}", ha="center", va="center", fontsize=12)
118
119 for (nome, (x, y)), T in zip(interiores.items(), t):
120     sc = ax.scatter(x, y, s=420, c=[T], cmap=cmap, vmin=0, vmax=1,
121                   edgecolors="k", zorder=3)
122     ax.text(x, y + 0.28, f"{nome} = {T:.4f}", ha="center", fontsize=11,
123           bbox=dict(facecolor="white", edgecolor="none", alpha=0.8, pad=1.5))
124
125 cb = fig.colorbar(sc, ax=ax, fraction=0.046, pad=0.04)
126 cb.set_label("temperatura")
127 ax.set_aspect("equal")
128 ax.set_xlim(-R - 0.4, R + 0.4)
129 ax.set_ylim(-R - 0.4, R + 0.4)
130 ax.axis("off")
131 ax.set_title("Placa circular: temperaturas na malha")
132 fig.tight_layout()
133 fig.savefig(arquivo, dpi=200)
134 plt.close(fig)
135
136 if __name__ == "__main__":
137     np.set_printoptions(precision=4, suppress=True)
138     M, b = montar_sistema()
139     A = np.eye(4) - M
140     print("M =\n", M, "\nb = ", b, "\nA = I - M =\n", A)
141
142     eh_pd, L = cholesky_com_teste(A)
143     print("\nA é positiva definida?", eh_pd)
144     t, metodo, L = resolver(A, b)
145     if L is not None:
146         print("L (Cholesky) =\n", L)
147         print("||L L^T - A||_max =", np.abs(L @ L.T - A).max())
148         print(f"\nTemperaturas (resolvido por {metodo}):")
149         for k, v in enumerate(t, start=1):
150             print(f"t{k} = {v:.6f}")
151         print("Verificação: ||t - (M t + b)||_inf =", np.abs(t - (M @ t + b)).max())
152
153     # contra-exemplo: o teste detecta uma matriz simétrica NÃO PD
154     B = np.array([[1.0, 2.0], [2.0, 1.0]])
155     print("\nTeste do detector com B = [[1,2],[2,1]] (autovalores 3 e -1):",
156           cholesky_com_teste(B)[0])
157
158     # caminho alternativo: matriz simétrica NÃO PD é resolvida por P^T LU
159     c = np.array([3.0, 3.0])

```

```

159     x, metodo, _ = resolver(B, c)
160     print(f"Sistema B x = {c} (B não PD) resolvido por {metodo}: x = {x} (esperado [1,
161           1]))")
162
163     desenhar(t)
164     print("\nFigura salva em q3_placa.png")

```

questao4.py

```

1  """
2  Questão 4 (teórica) - Matriz de Gram A = BT B.
3
4  A demonstração está no relatório. Este script apenas ILUSTRA numericamente
5  as duas propriedades para matrizes B aleatórias de vários formatos:
6      (a) A = AT
7      (b) xT A x = ||Bx||22 ≥ 0 para todo x
8  Também mostra o caso m < n, em que A é apenas semidefinida (existe x ≠ 0 com
9  Bx = 0, logo xT A x = 0), e usa o Cholesky com teste da questão 3 para mostrar
10 que, quando B tem colunas linearmente independentes, A é de fato definida positiva.
11 """
12
13 import numpy as np
14
15 from questao3 import cholesky_com_teste
16
17 def gram(B):
18     m, n = B.shape
19     A = np.zeros((n, n))
20     for i in range(n):
21         for j in range(n):
22             A[i, j] = B[:, i] @ B[:, j] # a_ij = <coluna i, coluna j>
23     return A
24
25 def vetor_no_nucleo(B):
26     """Para B (m x n) com m < n, encontra x ≠ 0 com Bx = 0 por eliminação
27     de Gauss (forma escalonada) - sem bibliotecas de álgebra linear."""
28     a = np.array(B, dtype=float)
29     m, n = a.shape
30     pivos, lin = [], 0
31     for col in range(n):
32         if lin == m:
33             break
34         p = lin + int(np.argmax(np.abs(a[lin:, col])))
35         if abs(a[p, col]) < 1e-12:
36             continue
37         a[[lin, p]] = a[[p, lin]]
38         a[lin] /= a[lin, col]
39         for r in range(m):
40             if r != lin:
41                 a[r] -= a[r, col] * a[lin]
42         pivos.append(col)
43         lin += 1
44     livre = next(c for c in range(n) if c not in pivos)
45     x = np.zeros(n)
46     x[livre] = 1.0
47     for r, c in enumerate(pivos):
48         x[c] = -a[r, livre]
49     return x
50
51 if __name__ == "__main__":
52     rng = np.random.default_rng(4)
53     for (m, n) in [(5, 3), (3, 3), (2, 4)]:
54         B = rng.normal(size=(m, n))
55         A = gram(B)
56         sim = np.abs(A - A.T).max()
57         menor = min((x @ A @ x) for x in rng.normal(size=(20000, n)))
58         print(f"B {m}x{n}: max|A - AT| = {sim:.1e} "
59               f"min xT A x (20000 x) = {menor:.4e} ")

```

```

63         f"Cholesky diz PD? {cholesky_com_teste(A)[0]}")
64     x = rng.normal(size=n)
65     print(f" conferindo x^T A x = ||Bx||^2: {x @ A @ x:.6f} = {(B @ x) @ (B @ x):.6f} ")
66     if m < n:
67         z = vetor_no_nucleo(B)
68         print(f" m < n: z = {np.round(z, 4)} tem Bz = {np.round(B @ z, 12)} "
69               f"e z^T A z = {z @ A @ z:.1e} (zero, a menos de arredondamento) -> A é só semidefinida")
70
71     # robustez do teste de Cholesky (Questão 3) sobre muitas matrizes de Gram:
72     # m < n -> A é singular (só semidefinida): o teste nunca deve dizer "PD";
73     # m >= n -> colunas independentes (com prob. 1): o teste sempre deve dizer "PD".
74     rng2 = np.random.default_rng(2026)
75     tentativas = 5000
76     pd_com_m_menor, pd_com_m_maior = 0, 0
77     for _ in range(tentativas):
78         n = int(rng2.integers(3, 8))
79         m = int(rng2.integers(1, n))          # 1 <= m < n
80         pd_com_m_menor += cholesky_com_teste(gram(rng2.normal(size=(m, n))))[0]
81         m = int(rng2.integers(n, n + 4))      # m >= n
82         pd_com_m_maior += cholesky_com_teste(gram(rng2.normal(size=(m, n))))[0]
83     print(f"\n{tentativas} sorteios com m < n : teste disse PD em {pd_com_m_menor} (esperado 0)")
84     print(f"{tentativas} sorteios com m >= n: teste disse PD em {pd_com_m_maior} (esperado {tentativas})")

```

Convenções e referências

As referências do tipo [Teorema 6.21, p. 458] remetem a R. L. Burden, J. D. Faires e A. M. Burden, *Análise Numérica*, 10^a ed., Cengage Learning (tradução brasileira). A Questão 3 segue a Seção 10.11 de H. Anton e C. Rorres, *Álgebra Linear com Aplicações*. Seguindo o livro [Definição 6.22, p. 459], uma matriz $A \in \mathbb{R}^{n \times n}$ é **positiva definida** (PD) quando é simétrica e $x^\top A x > 0$ para todo $x \neq 0$. Para cada linha i , denotamos $R_i = \sum_{j \neq i} |a_{ij}|$ (soma dos módulos fora da diagonal).

Implementação. Cada questão tem um arquivo próprio (`questao1.py`, ..., `questao4.py`), reproduzidos no Apêndice. Conforme as instruções da lista, *nenhuma* rotina pronta de álgebra linear foi usada (nada de `numpy.linalg` ou `scipy`): o NumPy serve apenas como contêiner de vetores/matrizes e para operações elementares (somas, produtos escalares, produto externo, operações entre linhas). Todos os algoritmos (eliminação, fatorações, substituições) estão escritos explicitamente. Como as operações vetorizadas do NumPy executam em C, os tempos absolutos da Questão 2 refletem também essa escolha, mas os três métodos usam as mesmas primitivas.